

Análise de corpos em queda com arrasto aerodinâmico quadrático

Guilherme de Medeiros Burkert¹, Patrick Guasti Vande Meerssche¹,
Richard Júnio Costa de Assis¹, Rômulo Tavares Maciel¹

¹ Centro de Ciências Computacionais – Universidade Federal do Rio Grande (FURG)
Caixa Postal 474 – 96.203-900 – Rio Grande – RS – Brasil

patrickgvml@gmail.com

Abstract. *This paper presents a comparative analysis between the explicit Euler method and the fourth-order Runge-Kutta (RK4) method for the numerical integration of Ordinary Differential Equations (ODEs). The methods are applied to an initial value problem modeling the free fall of a body subjected to quadratic aerodynamic drag. The algorithms were implemented in Python and evaluated based on their truncation errors relative to the exact analytical solution. The results demonstrate the superior accuracy and stability of the RK4 method, particularly with a refined integration step ($h = 0.05$), where the RK4 absolute error remained on the order of 10^{-7} while Euler reached errors on the order of 10^{-2} , highlighting the suitability of RK4 for non-linear dynamic systems.*

Resumo. *Este trabalho apresenta uma análise comparativa entre o método de Euler explícito e o método de Runge-Kutta de 4ª ordem (RK4) para a integração numérica de Equações Diferenciais Ordinárias (EDOs). Os métodos são aplicados a um problema de valor inicial que modela a queda livre de um corpo sujeito a arrasto aerodinâmico quadrático. Os algoritmos foram implementados em Python e avaliados com base em seus erros de truncamento em relação à solução analítica exata. Os resultados demonstram a superioridade de precisão e estabilidade do método RK4, especialmente com a adoção de um passo de integração refinado ($h = 0.05$), no qual o erro absoluto do RK4 permaneceu na ordem de 10^{-7} enquanto o método de Euler apresentou erros na ordem de 10^{-2} , evidenciando a adequação do RK4 para sistemas dinâmicos não-lineares.*

1. Introdução

A simulação de sistemas físicos e de engenharia frequentemente recai na formulação de Equações Diferenciais Ordinárias (EDOs) para modelar a evolução temporal de variáveis de estado. No entanto, muitos desses sistemas apresentam comportamentos não-lineares, dificultando ou inviabilizando a obtenção de soluções analíticas fechadas. Diante desse cenário, a integração numérica consolida-se como a abordagem padrão para a resolução de Problemas de Valor Inicial (PVIs) [1].

O método de Euler destaca-se como a formulação mais elementar, extrapolando a solução a partir da inclinação local. Contudo, sua precisão linear o torna vulnerável à propagação rápida de erros de truncamento. Em contrapartida, algoritmos da família Runge-Kutta, em especial a formulação clássica de quarta ordem (RK4), oferecem uma

estimativa ponderada das derivadas intermediárias do intervalo, mitigando consideravelmente as limitações de estabilidade [2].

Dessa forma, o presente trabalho tem como objetivo modelar a dinâmica de queda com arrasto aerodinâmico quadrático e avaliar computacionalmente o desempenho dos métodos de Euler e RK4. A análise busca validar as implementações em Python, quantificando empiricamente o comportamento dos métodos quando submetidos a uma malha de avanço temporal estrita.

2. Descrição do Problema

2.1. Modelagem Física

O cenário proposto consiste em analisar o perfil de velocidade de um corpo em queda livre submetido a uma força de arrasto aerodinâmico que é diretamente proporcional ao quadrado de sua velocidade. Pela Segunda Lei de Newton, a equação de movimento é regida por:

$$m \frac{dv}{dt} = mg - c_d v^2 \quad (1)$$

Assumindo a constante geométrica e de massa $\alpha = c_d/m$, a EDO não-linear de primeira ordem é padronizada na forma canônica:

$$\frac{dv}{dt} = g - \alpha v^2 = f(t, v) \quad (2)$$

Para a simulação, foram definidos os seguintes parâmetros de estado: aceleração da gravidade $g = 9,80665 \text{ m/s}^2$, fator de arrasto $\alpha = 0,1 \text{ m}^{-1}$ e a condição inicial de repouso $v(0) = 0 \text{ m/s}$. O domínio de integração temporal foi estipulado em $t \in [0, 5,85]$ segundos.

2.2. Solução Analítica

Para viabilizar a aferição da validade dos estados numéricos, utilizou-se a determinação do estado exato do sistema. A EDO do arrasto quadrático é separável, e sua integração resulta na seguinte solução analítica fechada:

$$v(t) = \sqrt{\frac{g}{\alpha}} \tanh(\sqrt{g\alpha} t) \quad (3)$$

Esta função descreve o crescimento amortecido da velocidade até a assíntota correspondente à velocidade terminal teórica do corpo, $v_{\text{term}} = \sqrt{g/\alpha} \approx 9,9027 \text{ m/s}$.

3. Metodologia

Para solucionar a Equação 2, foram implementados, por meio da linguagem python, dois métodos iterativos de passo único. A discretização do domínio obedece a um passo de tempo definida por $h = 0,05 \text{ s}$.

3.1. Método de Euler

O método explícito de Euler aproxima a solução assumindo que a derivada da função permanece constante em todo o subintervalo temporal. A formulação matemática obedece à série de Taylor truncada após o primeiro termo, resultando em convergência global $\mathcal{O}(h)$:

$$v_{n+1} = v_n + hf(t_n, v_n) \quad (4)$$

3.2. Método de Runge-Kutta de 4ª Ordem (RK4)

O método de Runge-Kutta de quarta ordem avalia a função derivada em quatro instantes distintos dentro do passo atual. A atualização de estado é calculada por uma média ponderada destas previsões, o que neutraliza os erros de truncamento e entrega erro global $\mathcal{O}(h^4)$:

$$k_1 = f(t_n, v_n) \quad (5)$$

$$k_2 = f\left(t_n + \frac{h}{2}, v_n + \frac{h}{2}k_1\right) \quad (6)$$

$$k_3 = f\left(t_n + \frac{h}{2}, v_n + \frac{h}{2}k_2\right) \quad (7)$$

$$k_4 = f(t_n + h, v_n + hk_3) \quad (8)$$

$$v_{n+1} = v_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (9)$$

4. Resultados e Discussão

A simulação computacional vetorizada varreu os primeiros 5,85 s de queda livre utilizando $h = 0,05$ s. Para fins de clareza e síntese, a Tabela 1 exhibe amostragens do estado do sistema a cada segundo, além do último instante da simulação. A saída numérica completa também apresenta os erros absolutos de cada método: o maior erro de Euler observado foi aproximadamente $9,17 \times 10^{-2}$ m/s (próximo de $t = 1,10$ s), enquanto o RK4 permaneceu na faixa de 10^{-7} m/s em todo o intervalo.

Tabela 1. Amostragem dos Estados de Velocidade Numérica x Analítica ($h = 0,05$ s)

Tempo (s)	V. Exata (m/s)	V. Euler (m/s)	V. RK4 (m/s)
0.00	0.0000	0.0000	0.0000
1.00	7.5013	7.5918	7.5013
2.00	9.5328	9.5794	9.5328
3.00	9.8509	9.8620	9.8509
4.00	9.8957	9.8978	9.8957
5.00	9.9019	9.9022	9.9019
5.85	9.9027	9.9027	9.9027

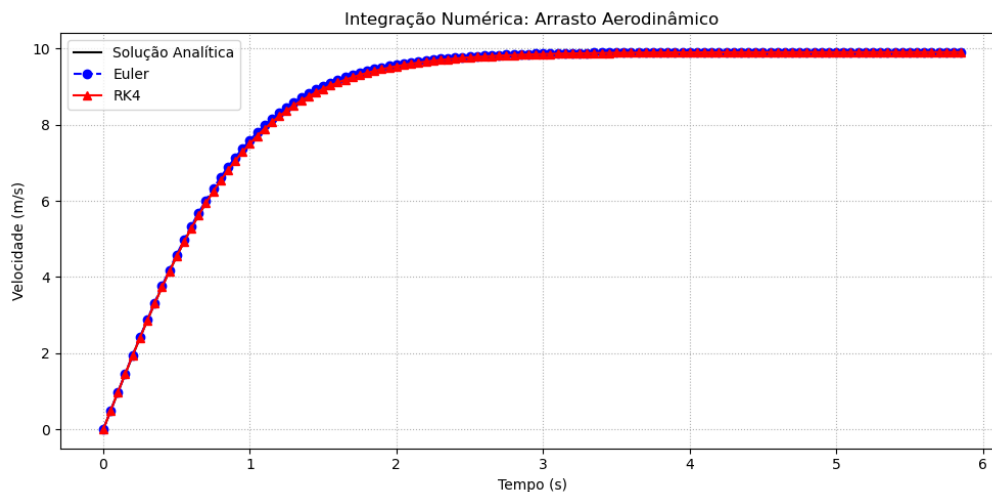


Figura 1. Comparação temporal da velocidade entre solução analítica, Euler e RK4.

Ao analisar a amostragem, constata-se visualmente a elevada fidelidade do RK4. Mesmo diante das não-linearidades do arrasto aerodinâmico submetido à velocidade crescente, a ponderação das quatro derivadas intermediárias assegurou convergência praticamente coincidente com a curva analítica, com erro absoluto na ordem de 10^{-7} .

A Figura 1 reforça esse comportamento ao mostrar a sobreposição entre a curva analítica e a solução numérica do RK4 ao longo de todo o intervalo de integração.

Por outro lado, o avanço puramente tangencial do método de Euler deixou resíduos perceptíveis ao longo de toda a fase de aceleração (zona transiente), culminando em um erro máximo de superestimação próximo a $t = 1,10\text{ s}$ ($\approx 9,17 \times 10^{-2}\text{ m/s}$ de desvio). A partir desse ponto, o erro de Euler passa a decrescer gradualmente, mas ainda permanece ordens de magnitude acima do erro do RK4 até o final da janela simulada.

5. Conclusão

A adoção de um passo de tempo restrito a $h = 0,05$ ressaltou a diferença existente entre integrações de primeira e de quarta ordem. A análise explícita dos erros mostrou que o RK4 permaneceu na ordem de 10^{-7} , enquanto o método de Euler atingiu erro máximo da ordem de 10^{-2} .

Conclui-se que o Método de Euler, embora fundamental do ponto de vista pedagógico, requer passos substancialmente menores para se aproximar da precisão observada no RK4, o que pode impor custo computacional adicional. O RK4, consequentemente, reitera seu posto como o algoritmo mais equilibrado entre custo e precisão para simulações ordinárias em engenharia.

Referências

- [1] BURDEN, R. L.; FAIRES, J. D.; BURDEN, A. M. **Numerical Analysis**. 10th ed. Boston: Cengage Learning, 2016.
- [2] ATKINSON, K. E. **An Introduction to Numerical Analysis**. 2nd ed. New York: John Wiley & Sons, 1989.

A. Código-Fonte Completo

<https://github.com/PatrickMeerssche/EDO-RK4-EULER>

Código 1. Simulação Numérica com Cálculo Explícito de Erros

```
import numpy as np
import matplotlib.pyplot as plt

def f(t, v):
    g = 9.80665
    alpha = 0.1
    return g - alpha * v**2

def sol_analitica(t):
    g = 9.80665
    alpha = 0.1
    return np.sqrt(g / alpha) * np.tanh(np.sqrt(g * alpha) * t)

t0, tf = 0.0, 5.85
h = 0.05
v0 = 0.0
t_vals = np.arange(t0, tf + h, h)
n_steps = len(t_vals)

v_euler = np.zeros(n_steps)
v_rk4 = np.zeros(n_steps)
v_exato = sol_analitica(t_vals)

v_euler[0] = v0
v_rk4[0] = v0

for i in range(n_steps - 1):
    t_n = t_vals[i]
    v_euler[i+1] = v_euler[i] + h * f(t_n, v_euler[i])
    k1 = f(t_n, v_rk4[i])
    k2 = f(t_n + h/2, v_rk4[i] + (h/2)*k1)
    k3 = f(t_n + h/2, v_rk4[i] + (h/2)*k2)
    k4 = f(t_n + h, v_rk4[i] + h*k3)
    v_rk4[i+1] = v_rk4[i] + (h/6) * (k1 + 2*k2 + 2*k3 + k4)

erro_euler = np.abs(v_euler - v_exato)
erro_rk4 = np.abs(v_rk4 - v_exato)

print(f"{'Tempo (s)':<10} | {'V. Exata':<10} | {'V. Euler':<10} |  
      | {'Erro Euler':<12} | {'V. RK4':<10} | {'Erro RK4'}")
print("-" * 80)
for i in range(n_steps):
    print(f"{t_vals[i]:<10.2f} | {v_exato[i]:<10.4f} |  
          {v_euler[i]:<10.4f} | {erro_euler[i]:<12.4e} |  
          {v_rk4[i]:<10.4f} | {erro_rk4[i]:.4e}")
```

```
plt.figure(figsize=(10, 5))
t_fine = np.linspace(t0, tf, 200)
plt.plot(t_fine, sol_analitica(t_fine), 'k-', label='Solucao
    Analitica')
plt.plot(t_vals, v_euler, 'bo--', label='Euler')
plt.plot(t_vals, v_rk4, 'r^--', label='RK4')
plt.title('Integracao Numerica: Arrasto Aerodinamico')
plt.xlabel('Tempo (s)')
plt.ylabel('Velocidade (m/s)')
plt.legend()
plt.grid(True, ls=':')
plt.tight_layout()

plt.figure(figsize=(8, 5))
v_plot = np.linspace(0, max(v_exato) * 1.1, 300)
f_plot = f(0, v_plot)
plt.plot(v_plot, f_plot, 'm-', linewidth=2, label='f(t, v) = g
    - alpha*v^2')
plt.title('Funcao Principal')
plt.xlabel('Velocidade v (m/s)')
plt.ylabel('f(t, v) (m/s2)')
plt.axhline(0, color='gray', linewidth=1, linestyle='--')
plt.legend()
plt.grid(True, ls=':')
plt.tight_layout()

plt.show()
```